Miscellaneous Examples

Example 1

create collections named demo having fields name,age,dep,salary,gender

1.insert 7 records into demo collection
2. find name and age of all person who are female
db.demo.find({gender:'female'},{name:true,age:true,_id:false})
3.update the sal of all by 5000
db.demo.updateMany({},{$inc:{sal:5000}})
4.arrange all records in asscending order of name
db.demo.find().sort({name:1})
5.update all person who are male into female
db.demo.updateMany({gender:'male'},{$set:{gender:"female"}})
6.delete all records whose salary are gt 70000
db.deleteMany({sal:{$gt:70000}})

Example 2
create a collection faculty having fields fname, lname, sub, exp, age, gender, sal

1. enter 10 records into fac table
2. find all fnames where abc occurs as a substring or as a separate word
db.fac.find({fname:/abc/},{fname:1,lname:1,_id:0})
3. update document if lname beginning with P update their sal by half
db.fac.updateMany({lname:/^P/i},{$mul:{sal:0.5}})
4. update first record by gender to others
db.fac.updateOne({gender:'others'})
5. count records who are teaching fsd-2 sub
db.fac.find({sub:'fsd-2'}).count()
6. update all record and insert current date to that record
db.fac.updateMany({},{$set:{date:new Date()}})
7. update sal of person whose sal less than 20000 to double
db.fac.updateMany({sal:{$lt:20000}},{$mul:{sal:2}})
8. update subject java to fsd-2 if java is not found insert using updateOne
db.fac.updateOne({sub:'java'},{$set:{sub:'fsd-2'}},{upsert:true})
9. delete all records and table
db.fac.deleteMany({}) or db.fac.drop()

Example 3
Write commands to perform following tasks on employee collection having fields having name,age & joiningDate:
(1) Update the name="Senior citizen" having age>60 years.
(2) Update the name="JKL" having age=20 years. Insert this record, if it is not found.

(3) Retire all employees by deleting senior citizens from collection.

```
>db.employee.updateMany({age:{$gt:60}},{$set:{name:”senior citizen”}})
>db.employee.updateMany({age:20},{$set:{name:”JKL”}},{upsert:true})
>db.employee.deleteMany({name:”senior citizen”})
```

Example 4

Write commands to perform following tasks on employee collection having fields name,age & joiningDate:

(1) Insert 3-4 records in collection.

(2) List all employees who joined before 1st January, 2010.

(3) Update the name of employee to "WWW" whose joiningDate is "05-05-2015"

```
>db.employee.insertMany({
name:"xyz",age:27,DOJ:ISODate("2014-07-03")},
{name:"pqr",age:35,DOJ:ISODate("2011-02-10")},
{name:"def",age:28,DOJ:ISODate("2009-08-25")},
{name:"abc",age:30,DOJ:ISODate("2008-05-15")})
>db.employee.find({DOJ:{$lt:ISODate("2010-01-01")}})
>db.employee.updateOne({DOJ:ISODate("2015-05-05")},{$set:{name:"WWW"}})
```

Example 5

Write commands to perform following tasks on employee collection having fields having name,age & joiningDate:

(1) Delete all records having joiningDate before 1st January, 2010.

(2) List all employees having age>50 years.

(3) List only 1st employee having age>60 years

```
>db.employee.deleteMany({ joiningDate: { $lt: ISODate("2010-01-01") } });
>db.employee.find({ age: { $gt: 50 } });
>db.employee.findOne({age:{$gt:60}})
```

Example 6

Write commands to perform following tasks on employee collection having fields having name,age & joiningDate:

(1) Count no. of employees having age>=60 years.

(2) List all employees in descending order of names having names "ABC", "PQR", "XYZ".

(3) List all employees whose age lies between 25 to 50 years excluding all rest of the fields.

```
>db.employee.count({ age: { $gte: 60 } });
>db.employee.find({ name: { $in: ["ABC", "PQR", "XYZ"] } }).sort({ name: -1 });
>db.employee.find({ age: { $gte: 25, $lte: 50 } }, { _id: 0, name: 1, age: 1 });
```

Example 7
Consider following student collection:
[
{_id:123433,name: "2DD", surname:"GGG", age:22},
{_id:123434,name: "LLL", surname:"RRR", age:2},
{_id:123435,name: "KKK", surname:"III", age:32},
{_id:123436,name: "ZZZ", surname:"TTTT", age:9}
]
Do as directed:
(1) List all students whose name starts by digit only.
(2) List all students whose surname has exactly 4 letters only.
(3) List only names of students from youngest to oldest.
(4) List all students whose name has 3-10 letters only. Don't allow digits & underscore

```
> db.student.find({ name: /^[0-9]/ });
> db.student.find({ surname: /^[A-Za-z]{4}$/ });
> db.student.find({}, { _id: 0, name: 1 }).sort({ age: 1 });
> db.student.find({ name: /^[A-Za-z]{3,10}$/ });
```

Example: 8
Map following SQL queries to MongoDB query:
(1) alter table people add joiningDate datetime
(2) alter table people drop column joiningDate
(3) select age,name from people where status="PH"
(4) select * from people where status!="PH"
(5) select name from people order by age desc

```
> db.people.insertMany({ name: "John", age: 30, joiningDate: ISODate("2022-01-01") });
> db.people.updateMany({}, { $unset: { joiningDate: "" } });
>db.people.find({ status: "PH" }, { _id: 0, age: 1, name: 1 });
> db.people.find({ status: { $ne: "PH" } });
> db.people.find({}, { _id: 0, name: 1 }).sort({ age: -1 });
```

Example:9
Map following SQL queries to MongoDB query:
(1) update employee set name="TTT" where age not in {12,33,44,55}
(2) select count(*) from employee where age<>23
(3) update employee set age=age+10

```
>db.employee.updateMany({ age: { $nin: [12, 33, 44, 55] } }, { $set: { name: "TTT" } });
> db.employee.count ({ age: { $ne: 23 } });
> db.employee.updateMany({}, { $inc: { age: 10 } });
```

Example:10

Write a MongoDB query to upsert a document in the products collection. The document should have productId (a string), name (a string), and price (a number). If a document with productId "P123" already exists, update its price to 29.99. If it does not exist, insert a new document with productId "P123", name "Gadget", and price 29.99.

```
db.products.updateOne(
  { productId: "P123" },
  {
    $set: { name: "Gadget", price: 29.99 },
  },
  { upsert: true }
);
```

Example:11

Write a MongoDB query to update the status field to "inactive" for all documents in the subscriptions collection where the expiryDate is less than today's date and status is currently "active". Use the appropriate operators to perform this update.

```
db.subscriptions.updateMany(
  {
    expiryDate: { $lt: new Date() },
    status: "active"
  },
  { $set: { status: "inactive" } }
);
```

Example: 12

Find a person by their name in a MongoDB database and update their age using the findByIdAndUpdate method, returning the updated document.

```
const mg = require("mongoose");
mg.connect("mongodb://127.0.0.1:27017/lju1")
  .then(() => { console.log("success"); })
  .catch((err) => { console.error(err); });
  const personSchema = new mg.Schema({
```

```
    name: String,
    age: Number,
    active: Boolean
  });
  const Person = mg.model("Person", personSchema);

const updatefunction = async () => {
 try {
  // Step 1: Find the person by name

  const person = await Person.findOne({ name:"test" });

  if (person) {
   const personId = person._id;

   // Step 2: Use findByIdAndUpdate to update the age
   const updatedPerson = await Person.findByIdAndUpdate(
    personId,
    { age: 50 },
    { new: true }
   );
   console.log("Updated Person:", updatedPerson);
  } else {
   console.log("Person not found");
  }
 } catch (err) {
  console.error("Error updating person's branch:", err);
 }};
updatefunction();
```

Example: 13

Create a Node.js program using Mongoose to demonstrate basic CRUD operations on a MongoDB collection named Person. The program should:

1. **Insert multiple documents** into the collection.
2. **Update** a document using a field-based filter (updateOne).
3. **Update** another document using its _id (findByIdAndUpdate).
4. **Delete** a document using its _id (findByIdAndDelete).

```
const mongoose = require("mongoose");

// 1. Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/lju")
  .then(() => console.log("Connected to MongoDB"))
  .catch(err => console.error("Connection error:", err));

// 2. Define Schema and Model
const personSchema = new mongoose.Schema({
  name: String,
  age: Number,
  active: Boolean
});

const Person = mongoose.model("Person", personSchema);

// 3. Main Function
async function runAllOperations() {
  try {
    // Insert multiple documents
    const inserted = await Person.insertMany([
      { name: "daya", age: 25, active: true },
      { name: "roshan", age: 28, active: false },
      { name: "babita", age: 35, active: true }
    ]);
    console.log("Inserted:", inserted);

    // Update one document using updateOne
    const updateResult = await Person.updateOne(
      { name: "daya" },
      { $set: { age: 30 } }
    );
    console.log("updateOne result:", updateResult);

    // Update document by ID
    const uid = await Person.findOne({ name: "roshan" });
    if (uid) {
      const updatedJane = await Person.findByIdAndUpdate(
        uid._id,
        { age: 32 },
        { new: true }
      );
      console.log("Updated by ID:", updatedJane);
    }

    // Delete document by ID
    const did = await Person.findOne({ name: "babita" });
    if (did) {
      const deletedoc = await Person.findByIdAndDelete(did._id);
```

```
    console.log("Deleted by ID:", deletedoc);
   }
  } catch (err) {
   console.error("Error:", err.message);
  }
}

// 4. Execute
runAllOperations();
```

Example:14

Write a program to design a feedback form (name, email, feedback (radio button options), comments, submit button) and save the submitted form data to database collection “feedback”. Now, by submitting a form display success message on next page "/feedback_data" and also give a link to display all the submitted feedbacks from the database collection on "/display_feedback" page .

e1.js

```
var expr = require("express");
var app = expr();
const mg = require("mongoose");

// Connect to MongoDB
mg.connect("mongodb://127.0.0.1:27017/lju")
  .then(() => { console.log("Successful") })
  .catch((err) => { console.error(err) });

mg.pluralize(null);
const myschema = new mg.Schema({
  uname: { type: String, required: true },
  email: { type: String, required: true },
  feedback: { type: String, required: true },
  comments: { type: String, required: true }
});

const person = mg.model("feedback", myschema);

app.use(expr.static(__dirname, { index: "e1.html" }));

// Insert data into MongoDB
app.get("/feedback_data", (req, res) => {
  const personData = new person({
```

```
    uname: req.query.uname,
    email: req.query.mail,
    feedback: req.query.feedback,
    comments: req.query.comments,
  });

  personData.save()
  res.send("Record inserted <a href='/display_feedback'>Display Feedback</a>")

});

// Fetch data from MongoDB and display in HTML table
app.get("/display_feedback", async(req, res) => {
  const f= await person.find();
  res.write(`<table border="1px
solid"><tr><th>Name</th><th>feddback</th><th>Comments</th></tr>`)
  f.map((f1)=>{

res.write(`<tr><td>${f1.uname}</td><td>${f1.feedback}</td><td>${f1.comments}</td></tr>
`)
  })
  res.write()
  res.send(`</table>`);
  });

app.listen(4001, () => {
  console.log("Server is running on port 4001");
});
```

e1.html

```
<html>
   <head><title>Feedback Form</title></head>
   <body>
      <form action="/feedback_data" method="get">
         Name: <input type="text" name="uname"/>
         Email: <input type="email" name="mail"/>
         Feedback: <select name="feedback">
            <option value="bad">Bad</option>
            <option value="good">Good</option>
            <option value="verygood">Very Good</option>
            <option value="excellent"SS>Excellent</option>
         </select>
         Comments: <textarea name="comments" rows="10" cols="15"
placeholder="Comments"></textarea>
```

```
        <input type="submit" value="Submit"/>
      </form>
   </body></html>
```

Two Way connection with React (*For Reference Only)

Create a form with login and signup facilities. If user is already registered than he can login if registered username and password matches with entered details. If u has not registered than he can signup with username and password and details should be entered in database. After login it should display that login successful.

Folder structure – Backend -> sererver.js

Frontend -> App.js, Login.js and Signup.js

Express Server.js

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const bcrypt = require('bcryptjs');
const cors = require('cors');

const app = express();

app.use(cors());
app.use(bodyParser.json());

mongoose.connect('mongodb://127.0.0.1:27017/PKP');

const UserSchema = new mongoose.Schema({
  username: String,
  password: String,
});

const User = mongoose.model('User', UserSchema);

app.post('/api/signup', async (req, res) => {
  try {
```

```
    const { username, email, password } = req.body;
    console.log("Usename is"+req.body.username);
    const hashedPassword = await bcrypt.hash(password, 10);
    const newUser = new User({ username, password: hashedPassword });
    await newUser.save();
    res.status(201).json({ message: 'User signed up successfully.' });
  } catch (error) {
    res.status(500).json({ error: 'An error occurred.' });
  }
});

app.post('/api/login', async (req, res) => {
  try {
    const { username, password } = req.body;
    const user = await User.findOne({ username });

    if (!user) {
      return res.status(401).json({ error: 'User not found.' });
    }

    const passwordMatch = await bcrypt.compare(password, user.password);

    if (!passwordMatch) {
      return res.status(401).json({ error: 'Invalid password.' });
    }

    res.json({ message: 'Login successful.' });
  } catch (error) {
    res.status(500).json({ error: 'Aa error aave che.' });
  }
});
app.listen(5000)
//const PORT = process.env.PORT || 3000;
//app.listen(PORT, () => {
//console.log(`Server is running on port ${PORT}`);
//});
```

React:

Signup2.jsx

```
import React, { useState } from 'react';
import axios from 'axios';

function Signup2() {
```

```
 const [username, setUsername] = useState('');
 const [password, setpassword] = useState('');
 const handleSignup = async (e) => {
  e.preventDefault();

  try {
   await axios.post('http://localhost:5000/api/signup', {
    username,
   password
   });
   alert('User signed up successfully.');
   setUsername('');
  setpassword('');
  }
  catch (error) {
     alert('An error occurred.');
  }
 };

 return (
  <div >
<h1>Sign Up</h1>
   <form onSubmit={handleSignup} method='post'>
   <label >Name</label>
<input
       type="text"
       value={username}
       required
       onChange={(e) => setUsername(e.target.value)}
      />
<label>Password</label>
<input
       type="password"
       value={password}
       required
       onChange={(e) => setpassword(e.target.value)}
      />

<button type="submit">Sign Up</button>
   </form>
  </div>
 );
}
```

```
export default Signup2;
```

Login2.jsx

```
import React, { useState } from 'react';
import axios from 'axios';


function Login2() {
  const [username, setUsername] = useState('');
  const [password, setpassword] = useState('');
  const handleLogin = async (e) => {
    e.preventDefault();

    try {
      await axios.post('http://localhost:5000/api/login', {
      username,
       password
      });
      alert('User Log in successfully.');
      setUsername('');
     setpassword('');
    }
    catch (error) {
        alert('Log in Failed.');
    }
  };

  return (
    <div >
<h1>Login</h1>
      <form onSubmit={handleLogin} method='post'>
      <label >Name</label>
<input
          type="text"
          value={username}
          required
          onChange={(e) => setUsername(e.target.value)}
        />
<label>Password</label>
<input
          type="password"
          value={password}
          required
          onChange={(e) => setpassword(e.target.value)}
        />

<button type="submit">Login</button>
      </form>
    </div>
  );
}
```

```
export default Login2;
```

*Import Signup2.jsx and Login2.jsx in App.jsx

```
 import React from "react";
import { BrowserRouter as Router, Routes, Route, Link } from "react-router-dom";
import Signup2 from "./Signup2";
import Login from "./Login";

function App() {
  return (
    <Router>
      <div>
        <nav>
          <ul>
            <li><Link to="/signup">Sign Up</Link></li>
            <li><Link to="/login">Login</Link></li>
          </ul>
        </nav>

        <Routes>
          <Route path="/signup" element={<Signup2 />} />
          <Route path="/login" element={<Login />} />
          <Route path="/" element={<h2>Welcome! Please select Sign Up or Login.</h2>} />
        </Routes>
      </div>
    </Router>
  );
}

export default App;
```

NODE-MONGOOSE (Reference)

Develop a Node.js program using Mongoose to:

1. **Create a single document** in a MongoDB collection.
2. **Update the same document multiple times**, triggering automatic version tracking (__v field) by Mongoose.
3. **Demonstrate how Mongoose increments the __v field** with each successful update to help manage optimistic concurrency.

```
const mg = require("mongoose");

mg.connect("mongodb://localhost:27017/lju")
  .then(() => console.log("MongoDB connected successfully"))
  .catch(err => console.error("Connection error:", err));

const mySchema = new mg.Schema(  {
    name: { type: String, required: true },
    Surname: String,
    age: Number,
    active: Boolean,
    date: { type: Date, default: () => new Date() }
  },
  {
    strict: false ,// Allow extra fields like "city"
    // versionKey defaults to true, so __v will work
    optimisticConcurrency:true
  });

// Optional: Track version conflicts
//mySchema.set('optimisticConcurrency', true);

const Person = mg.model("Student", mySchema);

// Step 1: Create a document
async function createDoc() {
  try {
    let personData = new Person({
      name: "test",
      Surname: "XYZ",
      age: 3,
      active: true,
      city: "Mumbai"
    });

    const created = await personData.save();
    console.log("Initial document:", created);

    // Step 2: Update it
    created.age = 4;
    const updated1 = await created.save(); // __v = 1
    console.log("After 1st update (__v):", updated1.__v);

    // Reload from DB
    const fetched1 = await Person.findById(updated1._id);
    console.log("Fetched after 1st update:", fetched1);

    fetched1.name = "abc";
    const updated2 = await fetched1.save(); // __v = 2
```

```
  console.log("After 2nd update (__v):", updated2.__v);

  // Reload again
  const fetched2 = await Person.findById(updated2._id);
  console.log("Fetched after 2nd update:", fetched2);
 } catch (err) {
  console.log("Error Occurred:", err.message);
 }}

createDoc();
```

Read Document React+express

SignUp

```
import React, { useState,useEffect } from 'react';
import axios from 'axios';

function Signup() {
 const [username, setUsername] = useState('');
 const [notes, setNotes] = useState([]);

useEffect(() => { fetchNotes(); }, []);

 async function fetchNotes() {
  const { data } = await axios.get("http://localhost:5000/note");
  setNotes(data);
 }

 const handleSignup = async (e) => {
  e.preventDefault();

  try {
   await axios.post('http://localhost:5000/signup', {username});
   alert('User signed up successfully.'+username);
  //  document.getElementById('test').innerHTML=username
    setUsername('');
     fetchNotes();
   }
 catch (error) {
    console.error('Error signing up:', error);
   alert('An error occurred.');
  }
 };
 return (
  <div>
```

```
    <h1>Sign Up</h1>
    <form onSubmit={handleSignup} method ='post'>
    Name: <input type="text" placeholder="Username" value={username}
       onChange={(e) => setUsername(e.target.value)} />
      <button type="submit">Sign Up</button>
    </form>
<h1 id='test'>{username}</h1>
<ul>
      {notes.map(n => (
        <li key={n._id}>{n.username}</li>
      ))}
    </ul>
  </div>  )}
export default Signup;
```

Signup.js

```
const express = require('express');
const mg = require('mongoose');
const cors = require('cors');
const app = express();

app.use(cors());
app.use(express.json());
mg.connect('mongodb://127.0.0.1:27017/User') .then(()=>{console.log("Connection
Success")})
const UserSchema = new mg.Schema({  username:String  });

const User = new mg.model('User', UserSchema);

app.post('/signup', async (req, res) => {
  try {
    const { username} = req.body;
//console.log("Username is" + req.body.username)

    const newUser = new User({ username});
    await newUser.save();
    res.send();
// or res.json({message:'Username inserted Successfully'})
  } catch (error) {
    res.send(error);
//or res.json({message:'Error in inserted Record'})
  }
});
```

```
app.get("/note", async (_req, res) => {
  const notes = await User.find().sort({ _id: -1 }).limit(10);
  res.json(notes);
});
app.listen(5000);
```

Array of objects (* Ref)

Example -1 Insert one document and update Status from Incomplete to Complete of taskID: "T002"

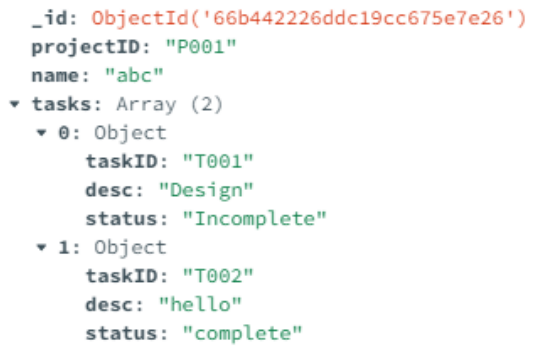
- db.stu.insertOne({projectID:"P001",name:"abc",
  tasks:[{taskID:"T001",desc:"Design",status:"Incomplete"},
  {taskID:"T002",desc:"development",status:"Incomplete"}
  ]})

To read second element of array use $arrayElemAt

- db.stu.find({}, { projectID: 1, name: 1,secondTask: { $arrayElemAt: ["$tasks", 1] }})

To Update

- db.stu.updateOne({projectID:"P001","tasks.taskID":"T002"},{$set:{"tasks.$.status":"complete"}})



Example -2 one document having name and project fields. project field have an array of strings. write a code to update project field with new value whose project filed have value of array as "abc"

- db.emp.insertOne({name:"abc",project:["asqsas","aseasdae"]})
- db.emp.updateOne({ name: "abc" },{ $set: { "project.0": "new project" } })